

Project: Simulated Annealing e Facility Location

ottimizzazione stocastica, paesaggi energetici e posizionamento di infrastrutture

1. Obiettivi della dispensa

Questa dispensa introduce il simulated annealing come caso di studio per un corso di metodi computazionali per modelli stocastici, applicato al problema del posizionamento ottimale di infrastrutture sul territorio italiano.

Gli obiettivi sono sei:

1. formalizzare il simulated annealing come processo stocastico di Markov a temperatura variabile;
2. motivare l'accettazione di mosse peggiorative come meccanismo di fuga dai minimi locali;
3. introdurre il problema della facility location e la sua formulazione come problema di ottimizzazione combinatoria;
4. applicare l'algoritmo a dati reali dei capoluoghi di provincia italiani per il posizionamento ottimale di ospedali o hub logistici;
5. studiare il ruolo del cooling schedule e confrontare diverse strategie di raffreddamento;
6. discutere la connessione tra simulated annealing, la distribuzione di Boltzmann e i processi MCMC.

Dal punto di vista del corso, questo progetto introduce l'unico algoritmo di tipo MCMC del set. Rispetto agli altri progetti, la stocasticità non è nella dinamica del fenomeno studiato, ma nell'algoritmo di ricerca: il rumore è uno strumento deliberato per esplorare uno spazio di soluzioni troppo grande per essere esplorato sistematicamente.

2. Motivazione: ottimizzazione su spazi enormi

2.1 Quando l'ottimizzazione esatta è impossibile

Molti problemi pratici richiedono di trovare la configurazione migliore tra un numero astronomico di alternative. Non si tratta di massimizzare una funzione continua derivabile — strumenti come il gradiente funzionano bene in quel caso.

Si tratta di scegliere tra oggetti discreti: quali k città tra 108 ospitano un ospedale di primo livello? In quale ordine un tecnico visita 50 sottostazioni elettriche? Come si dispongono 12 magazzini tra 300 comuni per minimizzare i costi di distribuzione?

Il numero di configurazioni possibili è tipicamente fattoriale o esponenziale nel numero di variabili. Per il problema di posizionare $k = 10$ ospedali tra 108 capoluoghi:

$$\binom{108}{10} = \frac{108!}{10! \cdot 98!} \approx 9.5 \times 10^{13}.$$

Quasi cento miliardi di configurazioni. Nessun computer può valutarle tutte. Servono euristiche.

2.2 Il problema dei minimi locali

La soluzione ovvia è la discesa greedy: si parte da una soluzione casuale, si cerca una mossa che la migliori, si accetta, si ripete. È veloce ma si ferma non appena non esistono mosse miglioranti — cioè in un minimo locale.

I minimi locali sono il problema fondamentale dell'ottimizzazione combinatoria. In problemi reali, il numero di minimi locali è enorme e la qualità del minimo locale trovato dipende fortemente dal punto di partenza. Il minimo globale è di solito molto meglio di un minimo locale tipico.

Esempio concreto. Si vogliono posizionare 5 ospedali tra i capoluoghi italiani. Un algoritmo greedy potrebbe convergere a una soluzione che mette tutti e cinque al Nord, perché la densità di popolazione è alta e ogni mossa locale (spostare un ospedale di una città) non migliora il costo. Ma una soluzione con distribuzione geografica più equilibrata sarebbe globalmente migliore. Per trovarla bisogna essere disposti ad accettare temporaneamente configurazioni peggiori.

2.3 L'idea del simulated annealing

Il simulated annealing (SA) risolve il problema dei minimi locali introducendo una componente stocastica controllata: con una certa probabilità, l'algoritmo accetta anche mosse che peggiorano la soluzione corrente.

La probabilità di accettare una mossa peggiorante non è costante: dipende da un parametro T chiamato **temperatura**, che decresce nel corso dell'algoritmo. All'inizio, con temperatura alta, quasi tutte le mosse vengono accettate: il sistema esplora liberamente lo spazio. Man mano che la temperatura scende, le mosse peggioranti vengono accettate sempre meno spesso: il sistema si "raffredda" e converge verso un minimo.

Il nome viene dalla metallurgia: il ricottura (annealing) di un metallo consiste nel portarlo ad alta temperatura e poi raffreddarlo lentamente. A temperatura alta gli atomi si muovono liberamente e trovano configurazioni energeticamente favorevoli; un raffreddamento troppo rapido “intrappola” gli atomi in configurazioni disordinate (vetro), mentre un raffreddamento lento permette di formare cristalli (stato di minima energia).

3. Il problema della facility location

3.1 Formulazione

Il problema della k -median facility location e' il seguente.

Dati:

- n siti (citta', comuni, nodi di una rete) con domanda w_i ciascuno;
- k strutture (ospedali, magazzini, hub) da posizionare tra i siti;
- una funzione di costo $d(i, j)$ tra ogni coppia di siti.

Si vuole trovare un sottoinsieme $S \subseteq \{1, \dots, n\}$ con $|S| = k$ che minimizzi il **costo totale ponderato**:

$$C(S) = \sum_{i=1}^n w_i \cdot \min_{j \in S} d(i, j).$$

Ogni sito i e' “assegnato” alla struttura piu' vicina in S , e il costo e' la distanza a quella struttura pesata per la domanda w_i .

Interpretazione sanitaria. I siti sono i 108 capoluoghi di provincia italiani. La domanda w_i e' la popolazione del capoluogo i . La struttura e' un ospedale di primo livello. Il costo $d(i, j)$ e' la distanza in km tra i capoluoghi i e j . Minimizzare $C(S)$ significa minimizzare il numero totale di km-persona che i cittadini devono percorrere per raggiungere l'ospedale piu' vicino.

3.2 Complessita' computazionale

Il problema k -median e' NP-hard per k generico. Non esiste un algoritmo polinomiale che lo risolva esattamente (salvo $P = NP$). Per questo si usano euristiche come SA, algoritmi genetici, o approssimazioni garantite.

3.3 Varianti

k -center (minimax). Invece di minimizzare la distanza media ponderata, si minimizza la distanza massima:

$$C_{\max}(S) = \max_{i=1}^n \min_{j \in S} d(i, j).$$

Garantisce che nessun cittadino sia a piu' di una certa distanza dall'ospedale piu' vicino. E' piu' equa del k -median ma puo' sacrificare l'efficienza media.

Capacita' limitata. Ogni struttura puo' servire al massimo Q unita' di domanda. Introduce il vincolo che un ospedale non sia sovraffollato.

Costo di apertura. Posizionare una struttura in certi siti e' piu' costoso che in altri (terreno, accessibilita'). Si aggiunge un termine di costo fisso al problema.

3.4 Dati reali: capoluoghi italiani

Per questo progetto si usano i 108 capoluoghi di provincia italiani con:

- coordinate geografiche (latitudine e longitudine);
- popolazione residente (ISTAT, ultimo censimento disponibile).

La distanza tra due capoluoghi si calcola con la **formula dell'emiseno (haversine)**:

$$d(i, j) = 2R \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta\phi}{2} \right) + \cos \phi_i \cos \phi_j \sin^2 \left(\frac{\Delta\lambda}{2} \right)} \right),$$

dove ϕ e' la latitudine in radianti, λ e' la longitudine, $\Delta\phi = \phi_j - \phi_i$, $\Delta\lambda = \lambda_j - \lambda_i$, e $R = 6371$ km e' il raggio terrestre medio.

Questa formula da' la distanza in linea d'aria. Per applicazioni reali si userebbe la distanza stradale, ma la distanza in km e' un ottimo proxy e ha il vantaggio di essere calcolabile analiticamente.

4. Il simulated annealing

4.1 Schema generale

Il simulated annealing e' un algoritmo iterativo. Lo stato corrente e' una soluzione S del problema di ottimizzazione. A ogni passo:

1. si genera una **mossa**: una piccola modifica casuale della soluzione corrente, che produce una soluzione candidata S' ;
2. si calcola la **variazione di costo**: $\Delta C = C(S') - C(S)$;
3. si **accetta o rifiuta** la mossa:
 - se $\Delta C \leq 0$ (la mossa migliora o non cambia il costo): si accetta sempre;
 - se $\Delta C > 0$ (la mossa peggiora il costo): si accetta con probabilita'

$$P(\text{accetta}) = e^{-\Delta C/T}.$$

4. si **aggiorna la temperatura** secondo il cooling schedule.

4.2 Il criterio di accettazione di Metropolis

La regola di accettazione

$$P(\text{accetta}) = \min\left(1, e^{-\Delta C/T}\right)$$

e' il **criterio di Metropolis**, originariamente introdotto nel 1953 per simulare sistemi fisici all'equilibrio termico.

Le proprieta' chiave sono:

- per $T \rightarrow \infty$: $e^{-\Delta C/T} \rightarrow 1$, tutte le mosse vengono accettate. Il sistema esplora casualmente senza preferire nessuna direzione.
- per $T \rightarrow 0$: $e^{-\Delta C/T} \rightarrow 0$ per $\Delta C > 0$, solo le mosse miglioranti vengono accettate. L'algoritmo diventa una discesa greedy.
- per T intermedia: le mosse peggioranti vengono accettate con probabilita' che decresce esponenzialmente con $\Delta C/T$. Mosse che peggiorano poco sono piu' facili da accettare di mosse che peggiorano molto.

Esempio concreto. Con $T = 100$ km e $\Delta C = 50$ km (la mossa sposta un ospedale aumentando il costo medio di 50 km per abitante): $P = e^{-50/100} = e^{-0.5} \approx 0.61$. La mossa viene accettata il 61% delle volte. Con $T = 10$: $P = e^{-5} \approx 0.007$. Quasi mai accettata. La temperatura controlla quanto il sistema e' disposto a "rischiare" di peggiorare.

4.3 Connessione con la distribuzione di Boltzmann

Il criterio di Metropolis non e' arbitrario. E' esattamente la regola di transizione che garantisce che, a temperatura fissa T , il processo di Markov converga alla distribuzione stazionaria di Boltzmann:

$$\pi_T(S) \propto e^{-C(S)/T}.$$

Questa distribuzione assegna probabilita' maggiore alle soluzioni con costo minore, con un'enfasi tanto piu' pronunciata quanto piu' bassa e' T .

Per $T \rightarrow 0$, la distribuzione di Boltzmann si concentra tutta sul minimo globale. Questo e' il senso preciso in cui SA "converge al minimo globale": se la temperatura venisse abbassata infinitamente lentamente (cooling schedule logaritmico), il processo convergerebbe con probabilita' 1 al minimo globale. In pratica si usa un raffreddamento piu' rapido, accettando una probabilita' non nulla di non trovare l'ottimo assoluto.

4.4 Il cooling schedule

Il cooling schedule specifica come decresce la temperatura nel corso dell'algoritmo. E' uno dei parametri piu' importanti e piu' delicati di SA.

Schema geometrico (esponenziale):

$$T_{n+1} = \alpha \cdot T_n, \quad 0 < \alpha < 1.$$

E' il piu' usato in pratica. Parametri tipici: $\alpha \in [0.90, 0.99]$. Con $\alpha = 0.95$ e $T_0 = 100$, dopo 100 iterazioni di raffreddamento si ha $T_{100} = 100 \cdot 0.95^{100} \approx 0.59$.

Schema lineare:

$$T_n = T_0 \left(1 - \frac{n}{N_{\max}} \right).$$

Semplice ma raffredda troppo rapidamente nelle prime fasi.

Schema logaritmico (teoricamente ottimale):

$$T_n = \frac{T_0}{\ln(1+n)}.$$

Garantisce convergenza al minimo globale ma e' troppo lento per applicazioni pratiche.

Reheat. In alcune varianti, se l'algoritmo sembra bloccato in un minimo locale, la temperatura viene temporaneamente riportata a un valore piu' alto (reheat). Questo permette di esplorare nuove regioni dello spazio.

4.5 Mosse per la facility location

Una mossa elementare per il problema k -median consiste in uno scambio:

- si estrae casualmente una struttura $j \in S$ da rimuovere;
- si estrae casualmente un sito $j' \notin S$ in cui aprire una nuova struttura;
- la nuova soluzione e' $S' = (S \setminus \{j\}) \cup \{j'\}$.

Questa mossa e' semplice, preserva il numero di strutture, e permette di raggiungere qualsiasi soluzione a partire da qualsiasi altra in un numero finito di passi (la catena di Markov e' irriducibile).

Il calcolo di ΔC puo' essere fatto in modo efficiente senza ricalcolare l'intera somma: basta aggiornare solo i siti che cambiano struttura di riferimento.

5. Analisi teorica: perche' SA funziona

5.1 SA come catena di Markov non omogenea

A ogni valore di temperatura T , SA definisce una catena di Markov sullo spazio delle soluzioni, con probabilita' di transizione determinate dal criterio di

Metropolis. Questa catena e' reversibile e ha come distribuzione stazionaria la distribuzione di Boltzmann π_T .

Quando la temperatura cambia, la catena cambia. SA e' quindi una **catena di Markov non omogenea**: i kernel di transizione variano nel tempo.

5.2 Condizione di convergenza

Il teorema di convergenza di SA afferma che, se il cooling schedule e' logaritmico

$$T_n \geq \frac{C^*}{\ln(1+n)},$$

dove C^* e' la profondita' massima di qualsiasi minimo locale (cioe' il massimo della differenza di costo tra un minimo locale e il minimo globale attraverso il miglior percorso possibile), allora l'algoritmo converge al minimo globale con probabilita' 1.

In pratica, C^* non e' noto, il cooling logaritmico e' troppo lento, e ci si accontenta di cooling geometrico con buoni risultati empirici.

5.3 Bilanciamento tra esplorazione e sfruttamento

SA formalizza matematicamente il trade-off tra esplorazione (exploration) e sfruttamento (exploitation) che compare in molti altri modelli del corso:

- a temperatura alta: esplorazione dominante, il sistema non sfrutta le informazioni accumulate;
- a temperatura bassa: sfruttamento dominante, il sistema converge verso il minimo trovato;
- a temperatura intermedia: bilanciamento ottimale.

Questo e' esattamente lo stesso trade-off del modello di March (exploration vs exploitation nelle organizzazioni), del bandit problem, e del problema del foraging.

6. La distribuzione reale degli ospedali italiani come benchmark

Un aspetto molto istruttivo del progetto e' il confronto tra la soluzione trovata da SA e la distribuzione reale degli ospedali di primo livello in Italia (DEA di II livello, circa 120 strutture).

La distribuzione reale riflette vincoli storici, politici e demografici che il modello non cattura: ospedali costruiti decenni fa in localita' che non erano allora capoluogo, vincoli regionali di autonomia sanitaria, eredita' di strutture ospedaliere storiche.

Confrontare la soluzione ottimale del modello con la realtà permette di rispondere a domande concrete: dove ci sono “sovradotazioni” (strutture ravvicinate)? Dove ci sono “carenze” (aree lontane da qualsiasi struttura)? Quale sarebbe il risparmio in termini di km-persona se si adottasse la distribuzione ottimale?

Questo trasforma il laboratorio computazionale in un esercizio di analisi di politica sanitaria.

7. Domande scientifiche che il modello permette di studiare

Il modello è utile per affrontare domande precise.

1. Come dipende la qualità della soluzione trovata dal cooling schedule? Quali valori di α e T_0 danno i migliori risultati?
2. Quanto spesso SA trova il minimo globale su questo problema? Come varia con il numero di iterazioni?
3. Come cambia la soluzione ottimale al variare di k ? Esiste un valore di k oltre il quale aggiungere strutture produce guadagni marginali minimi?
4. Come differisce la soluzione k -median (minimizza la distanza media) dalla soluzione k -center (minimizza la distanza massima)?
5. Quanto conta la scelta della soluzione iniziale? Una soluzione iniziale intelligente (ad esempio, basata su clustering geografico) porta a soluzioni migliori?
6. Qual è il costo della distribuzione reale degli ospedali rispetto all’ottimo calcolato? Dove sono le principali inefficienze geografiche?

8. Schema del laboratorio

8.1 Laboratorio 1 - Implementazione e visualizzazione

Obiettivo

Implementare SA per il problema k -median e visualizzare l’evoluzione dell’algoritmo.

Attività

1. caricare i dati dei 108 capoluoghi (coordinate e popolazione);
2. calcolare la matrice delle distanze haversine 108×108 ;
3. implementare SA con cooling geometrico;
4. eseguire SA con $k = 5$ e visualizzare la soluzione sul territorio italiano;
5. tracciare la curva del costo in funzione del numero di iterazioni.

Domande guida

- la curva del costo decresce in modo regolare o mostra salti verso l'alto?
- i salti verso l'alto corrispondono all'accettazione di mosse peggioranti?
- la soluzione con $k = 5$ e' geograficamente distribuita su tutto il territorio?

Output richiesto

- codice sorgente;
- mappa delle strutture posizionate con i bacini di utenza;
- curva del costo vs iterazioni con indicazione della temperatura;
- commento sulla distribuzione geografica.

8.2 Laboratorio 2 - Ruolo del cooling schedule

Obiettivo

Confrontare diverse strategie di raffreddamento e studiarne l'effetto sulla qualita' della soluzione.

Attivita'

1. fissare $k = 10$ e confrontare cooling con $\alpha = 0.99$, $\alpha = 0.95$, $\alpha = 0.80$, e cooling lineare;
2. per ogni schedule, eseguire 20 run indipendenti;
3. confrontare la distribuzione dei costi finali;
4. confrontare il tempo di calcolo.

Domande guida

- quale schedule trova soluzioni migliori in media?
- quale schedule e' piu' stabile (bassa varianza tra run)?
- esiste un trade-off tra qualita' e tempo di calcolo?

Output richiesto

- boxplot dei costi finali per i diversi schedule;
- tabella di costo medio, costo minimo, deviazione standard;
- commento sul trade-off qualita'/tempo.

8.3 Laboratorio 3 - k ottimale e curve di costo marginale

Obiettivo

Studiare come il costo ottimale varia con k e identificare il punto di rendimenti marginali decrescenti.

Attivita'

1. eseguire SA per $k = 1, 2, 3, 5, 8, 10, 15, 20$;
2. costruire la curva di costo ottimale in funzione di k ;
3. calcolare il costo marginale di aggiungere la k -esima struttura;
4. confrontare la curva k -median con la curva k -center.

Domande guida

- a quale k il costo marginale diventa piccolo?
- la curva di costo è convessa in k ?
- k -median e k -center concordano su quale sia la soluzione ottimale per k piccolo?

Output richiesto

- grafico del costo ottimale vs k ;
- grafico del costo marginale vs k ;
- mappe comparate delle soluzioni k -median e k -center per lo stesso k .

8.4 Laboratorio 4 - Confronto con la distribuzione reale

Obiettivo

Valutare la distribuzione reale degli ospedali di primo livello in Italia usando il modello come benchmark.

Attivita'

1. costruire una soluzione “reale” approssimata basata sui capoluoghi con ospedali di I livello;
2. calcolare il suo costo k -median;
3. confrontare con la soluzione ottimale per lo stesso k ;
4. identificare i capoluoghi con la maggiore discrepanza tra copertura reale e ottimale.

Domande guida

- quanto è efficiente la distribuzione reale rispetto all'ottimo?
- ci sono regioni sistematicamente sotto-servite?
- quali spostamenti di strutture porterebbero al maggior miglioramento?

Output richiesto

- mappa comparata della distribuzione reale e ottimale;
- tabella dei capoluoghi più distanti dall'ospedale più vicino;
- stima del risparmio potenziale in km-persona.

9. Una possibile estensione teorica

9.1 SA come campionamento MCMC

Il legame tra SA e i metodi Monte Carlo per catene di Markov (MCMC) e' profondo. A temperatura fissa T , SA e' un campionario MCMC dalla distribuzione di Boltzmann. Questo rende SA un caso speciale di un framework molto generale che comprende l'algoritmo di Metropolis-Hastings, il campionamento di Gibbs, e la Hamiltonian Monte Carlo.

In questa prospettiva, SA non e' solo un ottimizzatore ma anche uno strumento per esplorare e caratterizzare lo spazio delle soluzioni. Campionando molte soluzioni a una data temperatura, si puo' stimare la distribuzione dei costi e capire quanti minimi locali di buona qualita' esistono.

9.2 Algoritmi genetici: un confronto

Gli algoritmi genetici (GA) sono un'altra famiglia di metodi di ottimizzazione stocastica. Invece di lavorare su una singola soluzione come SA, lavorano su una popolazione di soluzioni che si combinano e mutano ad ogni generazione.

Il confronto SA vs GA e' molto istruttivo: SA e' piu' semplice, ha una teoria piu' solida, e funziona bene per spazi di soluzioni con struttura regolare. GA gestisce meglio problemi altamente non lineari e con struttura modulare. Per il problema k -median, SA e' generalmente preferibile.

10. Perche' questo e' un buon case study per il corso

Questo progetto e' particolarmente adatto a un corso di metodi computazionali per modelli stocastici per almeno cinque ragioni.

Primo, introduce il campionamento MCMC come strumento computazionale, che e' uno dei metodi piu' usati in tutta la scienza computazionale e non e' presente in nessun altro progetto del corso.

Secondo, il problema applicativo e' immediatamente comprensibile e politicamente rilevante: dove costruire ospedali, magazzini, centrali elettriche sono domande reali con impatto diretto sulla qualita' della vita.

Terzo, i dati sono reali e scaricabili: questo trasforma il laboratorio in un esercizio di analisi dati genuino, non solo una simulazione astratta.

Quarto, il ruolo della temperatura e del cooling schedule e' intuitivo e sperimentabile direttamente: lo studente "sente" la differenza tra un raffreddamento troppo rapido e uno troppo lento guardando le curve di costo.

Quinto, il tema del trade-off tra esplorazione e sfruttamento collega il progetto in modo molto diretto a March, Foraging, bandit problem e SGD, creando una coerenza tematica trasversale all'intero corso.

11. Conclusione

Il simulated annealing mostra come la stocasticita' possa essere uno strumento deliberato invece che una proprieta' del sistema studiato. Il rumore non e' qualcosa da controllare o ridurre: e' il meccanismo che permette all'algoritmo di sfuggire ai minimi locali e di esplorare lo spazio delle soluzioni.

La connessione con la fisica statistica — la distribuzione di Boltzmann, il concetto di temperatura, il raffreddamento lento come percorso verso il minimo energetico — non e' solo una metafora: e' una corrispondenza matematica precisa che garantisce le proprieta' di convergenza dell'algoritmo.

Dal punto di vista metodologico, il progetto combina in modo naturale:

- ottimizzazione combinatoria NP-hard;
- algoritmo MCMC a temperatura variabile;
- dati geografici reali;
- analisi del cooling schedule;
- confronto con benchmark empirici;
- interpretazione di politica sanitaria o infrastrutturale.

Il messaggio concettuale piu' importante e' che accettare deliberatamente soluzioni peggiori — con la probabilita' giusta, per il tempo giusto — e' spesso il modo piu' efficace per trovare soluzioni migliori.

12. Bibliografia minima

1. Kirkpatrick, S., Gelatt, C. D., and Vecchi, M. P. (1983). Optimization by Simulated Annealing. *Science*, 220(4598), 671-680.
2. Cerny, V. (1985). Thermodynamical Approach to the Traveling Salesman Problem: An Efficient Simulation Algorithm. *Journal of Optimization Theory and Applications*, 45(1), 41-51.
3. Hajek, B. (1988). Cooling Schedules for Optimal Annealing. *Mathematics of Operations Research*, 13(2), 311-329.
4. Arya, V., Garg, N., Khandekar, R., Meyerson, A., Munagala, K., and Pandit, V. (2004). Local Search Heuristics for k -Median and Facility Location Problems. *SIAM Journal on Computing*, 33(3), 544-562.
5. Robert, C. P., and Casella, G. (2004). *Monte Carlo Statistical Methods*. Springer.

Appendice A. Implementazione in Python: struttura del codice

Questa appendice propone una struttura semplice per implementare in Python SA per il problema k -median sui capoluoghi di provincia italiani.

Il codice e' volutamente elementare:

- poche librerie;
- funzioni corte;
- passaggi espliciti;
- nomi leggibili.

A.1 Librerie minime

```
import random
import math
import statistics
import matplotlib.pyplot as plt
```

A.2 Dati: capoluoghi di provincia italiani

```
# Ogni voce: (nome, regione, latitudine, longitudine, popolazione)
CAPOLUOGHI = [
    ('Torino', 'Piemonte', 44.4056, 7.6872, 847287),
    ('Vercelli', 'Piemonte', 45.3167, 8.4167, 44887),
    ('Novara', 'Piemonte', 45.4458, 8.6222, 103486),
    ('Cuneo', 'Piemonte', 44.3833, 7.5500, 57156),
    ('Asti', 'Piemonte', 44.9000, 8.2000, 75702),
    ('Alessandria', 'Piemonte', 44.9167, 8.6167, 86704),
    ('Biella', 'Piemonte', 45.5631, 8.0544, 43534),
    ('Verbania', 'Piemonte', 45.9233, 8.5522, 30378),
    ('Aosta', 'Valle d Aosta', 45.7372, 7.3197, 34062),
    ('Genova', 'Liguria', 44.4056, 8.9463, 565752),
    ('Imperia', 'Liguria', 43.8855, 8.0331, 41892),
    ('Savona', 'Liguria', 44.3069, 8.4814, 60724),
    ('La Spezia', 'Liguria', 44.1025, 9.8241, 91754),
    ('Milano', 'Lombardia', 45.4642, 9.1900, 1371498),
    ('Varese', 'Lombardia', 45.8206, 8.8257, 79584),
    ('Como', 'Lombardia', 45.8103, 9.0861, 84658),
    ('Sondrio', 'Lombardia', 46.1697, 9.8733, 21601),
    ('Bergamo', 'Lombardia', 45.6981, 9.6773, 122349),
    ('Brescia', 'Lombardia', 45.5416, 10.2118, 200465),
    ('Pavia', 'Lombardia', 45.1847, 9.1582, 72773),
    ('Cremona', 'Lombardia', 45.1333, 10.0333, 71606),
    ('Mantova', 'Lombardia', 45.1564, 10.7914, 48672),
    ('Lecco', 'Lombardia', 45.8553, 9.3903, 47469),
```

('Lodi',	'Lombardia',	45.3167,	9.5000,	45478),
('Monza',	'Lombardia',	45.5845,	9.2744,	123598),
('Bolzano',	'Trentino-Alto Adige',	46.4981,	11.3548,	108245),
('Trento',	'Trentino-Alto Adige',	46.0748,	11.1217,	120875),
('Verona',	'Veneto',	45.4386,	10.9928,	260125),
('Vicenza',	'Veneto',	45.5467,	11.5472,	112880),
('Belluno',	'Veneto',	46.1399,	12.2170,	35403),
('Treviso',	'Veneto',	45.6669,	12.2431,	84669),
('Venezia',	'Veneto',	45.4408,	12.3155,	249961),
('Padova',	'Veneto',	45.4064,	11.8768,	210440),
('Rovigo',	'Veneto',	45.0703,	11.7897,	50005),
('Udine',	'Friuli-Venezia Giulia',	46.0644,	13.2356,	98034),
('Gorizia',	'Friuli-Venezia Giulia',	45.9408,	13.6219,	34065),
('Trieste',	'Friuli-Venezia Giulia',	45.6522,	13.7722,	200713),
('Pordenone',	'Friuli-Venezia Giulia',	45.9564,	12.6611,	50335),
('Bologna',	'Emilia-Romagna',	44.4939,	11.3428,	419663),
('Ferrara',	'Emilia-Romagna',	44.8381,	11.6197,	130992),
('Modena',	'Emilia-Romagna',	44.6468,	10.9255,	190758),
('Reggio Emilia',	'Emilia-Romagna',	44.6989,	10.6298,	174378),
('Parma',	'Emilia-Romagna',	44.8015,	10.3278,	200081),
('Piacenza',	'Emilia-Romagna',	45.0522,	9.6933,	102194),
('Ravenna',	'Emilia-Romagna',	44.4175,	12.2011,	158739),
('Forli',	'Emilia-Romagna',	44.2228,	12.0408,	118216),
('Rimini',	'Emilia-Romagna',	44.0594,	12.5683,	158784),
('Firenze',	'Toscana',	43.7711,	11.2486,	367150),
('Prato',	'Toscana',	43.8808,	11.0966,	198031),
('Pistoia',	'Toscana',	43.9331,	10.9167,	92895),
('Lucca',	'Toscana',	43.8430,	10.5078,	89872),
('Pisa',	'Toscana',	43.7228,	10.4017,	91104),
('Livorno',	'Toscana',	43.5483,	10.3106,	152591),
('Arezzo',	'Toscana',	43.4636,	11.8797,	100012),
('Siena',	'Toscana',	43.3186,	11.3307,	53901),
('Grosseto',	'Toscana',	42.7594,	11.1133,	82087),
('Massa',	'Toscana',	44.0353,	10.1408,	67502),
('Perugia',	'Umbria',	43.1119,	12.3886,	162621),
('Terni',	'Umbria',	42.5636,	12.6483,	108035),
('Ancona',	'Marche',	43.6158,	13.5189,	99470),
('Pesaro',	'Marche',	43.9103,	12.9133,	94969),
('Macerata',	'Marche',	43.2989,	13.4531,	41540),
('Ascoli Piceno',	'Marche',	42.8539,	13.5747,	47322),
('Fermo',	'Marche',	43.1597,	13.7186,	36325),
('Roma',	'Lazio',	41.8933,	12.4828,	2751755),
('Viterbo',	'Lazio',	42.4178,	12.1047,	67726),
('Rieti',	'Lazio',	42.4047,	12.8622,	46946),
('Latina',	'Lazio',	41.4678,	12.9036,	126886),
('Frosinone',	'Lazio',	41.6397,	13.3447,	44744),

```

('L Aquila',      'Abruzzo',      42.3500, 13.3997, 69753),
('Teramo',        'Abruzzo',      42.6583, 13.7044, 53564),
('Pescara',       'Abruzzo',      42.4608, 14.2153, 116287),
('Chieti',        'Abruzzo',      42.3511, 14.1675, 47832),
('Campobasso',    'Molise',       41.5597, 14.6561, 48592),
('Isernia',       'Molise',       41.5994, 14.2300, 21567),
('Napoli',        'Campania',     40.8358, 14.2488, 909048),
('Caserta',       'Campania',     41.0739, 14.3328, 75500),
('Salerno',       'Campania',     40.6823, 14.7681, 131861),
('Avellino',      'Campania',     40.9147, 14.7906, 53570),
('Benevento',     'Campania',     41.1297, 14.7825, 58627),
('Bari',          'Puglia',       41.1253, 16.8667, 315606),
('Foggia',        'Puglia',       41.4578, 15.5447, 143618),
('Taranto',       'Puglia',       40.4764, 17.2283, 185162),
('Brindisi',      'Puglia',       40.6328, 17.9414, 84990),
('Lecce',         'Puglia',       40.3564, 18.1753, 93930),
('Barletta',      'Puglia',       41.3153, 16.2819, 93349),
('Potenza',       'Basilicata',    40.6394, 15.8019, 66034),
('Matera',        'Basilicata',    40.6667, 16.6000, 59776),
('Catanzaro',     'Calabria',     38.9097, 16.5878, 87639),
('Cosenza',       'Calabria',     39.3000, 16.2500, 64813),
('Reggio Calabria', 'Calabria',     38.1103, 15.6472, 172707),
('Crotone',       'Calabria',     39.0817, 17.1289, 61536),
('Vibo Valentia', 'Calabria',     38.6758, 16.1006, 32968),
('Palermo',       'Sicilia',     38.1111, 13.3522, 636872),
('Catania',       'Sicilia',     37.5023, 15.0873, 298957),
('Messina',       'Sicilia',     38.1931, 15.5500, 221613),
('Agrigento',     'Sicilia',     37.3111, 13.5765, 58887),
('Caltanissetta', 'Sicilia',     37.4906, 14.0600, 61525),
('Enna',          'Sicilia',     37.5650, 14.2756, 26966),
('Ragusa',        'Sicilia',     36.9281, 14.7314, 72960),
('Siracusa',      'Sicilia',     37.0755, 15.2866, 117863),
('Trapani',       'Sicilia',     38.0175, 12.5111, 67433),
('Cagliari',      'Sardegna',     39.2239, 9.1219, 154083),
('Sassari',       'Sardegna',     40.7268, 8.5606, 120836),
('Nuoro',         'Sardegna',     40.3217, 9.3311, 34713),
('Oristano',      'Sardegna',     39.9033, 8.5919, 30565),
('Sud Sardegna', 'Sardegna',     39.5533, 9.0131, 93619),
]

```

A.3 Distanza haversine

```

def haversine(lat1, lon1, lat2, lon2):
    R = 6371.0

    phi1 = math.radians(lat1)

```

```

phi2 = math.radians(lat2)
dphi = math.radians(lat2 - lat1)
dlam = math.radians(lon2 - lon1)

a = (math.sin(dphi / 2.0) ** 2
      + math.cos(phi1) * math.cos(phi2) * math.sin(dlam / 2.0) ** 2)

return 2.0 * R * math.asin(math.sqrt(a))

```

A.4 Precalcolo della matrice delle distanze

```

def build_distance_matrix(cities):
    n = len(cities)
    dist = [[0.0] * n for _ in range(n)]

    for i in range(n):
        for j in range(i + 1, n):
            d = haversine(cities[i][2], cities[i][3],
                          cities[j][2], cities[j][3])
            dist[i][j] = d
            dist[j][i] = d

    return dist

```

Il precalcolo e' fondamentale: la matrice 108×108 ha 11664 elementi, e ogni run di SA esegue migliaia di valutazioni del costo. Ricalcolare le distanze ad ogni passo rallenterebbe l'algoritmo di ordini di grandezza.

A.5 Funzione di costo k-median

```

def kmedian_cost(facilities_set, populations, dist):
    total = 0.0
    n = len(populations)

    for i in range(n):
        min_dist = min(dist[i][j] for j in facilities_set)
        total += populations[i] * min_dist

    return total

```

A.6 Aggiornamento incrementale del costo

Per efficienza, invece di ricalcolare l'intera somma dopo ogni mossa, si aggiorna solo la parte che cambia.

```

def kmedian_cost_delta(facilities_set, remove_idx, add_idx,
                      populations, dist):

```

```

delta = 0.0
n = len(populations)

for i in range(n):
    old_min = min(dist[i][j] for j in facilities_set)

    new_set = (facilities_set - {remove_idx}) | {add_idx}
    new_min = min(dist[i][j] for j in new_set)

    delta += populations[i] * (new_min - old_min)

return delta

```

Nota: per semplicità questa funzione ricalcola i minimi da zero. Una versione ottimizzata manterrebbe i minimi per ogni sito e li aggiornasse solo per i siti che cambiano struttura di riferimento.

A.7 Soluzione iniziale casuale

```

def random_initial_solution(n, k):
    indices = list(range(n))
    random.shuffle(indices)
    return set(indices[:k])

```

Una soluzione iniziale più intelligente è il **greedy sequenziale**: si aggiunge una struttura alla volta, scegliendo quella che riduce maggiormente il costo. Questo può migliorare la qualità della soluzione finale.

A.8 Il simulated annealing

```

def simulated_annealing(k, populations, dist,
                        T_start=1e7, T_end=1e2,
                        alpha=0.995, steps_per_temp=100,
                        seed=None):
    if seed is not None:
        random.seed(seed)

    n = len(populations)
    all_indices = list(range(n))

    # soluzione iniziale
    current = random_initial_solution(n, k)
    current_cost = kmedian_cost(current, populations, dist)

    best = set(current)
    best_cost = current_cost

```

```

T = T_start
cost_history = [current_cost]
temp_history = [T]

while T > T_end:
    for _ in range(steps_per_temp):
        # genera una mossa: rimuovi una struttura, aggiungine un'altra
        remove_idx = random.choice(list(current))

        candidates = [i for i in all_indices if i not in current]
        if not candidates:
            continue
        add_idx = random.choice(candidates)

        # calcola la variazione di costo
        delta = kmedian_cost_delta(current, remove_idx, add_idx,
                                   populations, dist)

        # criterio di Metropolis
        if delta <= 0.0:
            accept = True
        else:
            accept = random.random() < math.exp(-delta / T)

        if accept:
            current = (current - {remove_idx}) | {add_idx}
            current_cost += delta

            if current_cost < best_cost:
                best = set(current)
                best_cost = current_cost

        cost_history.append(current_cost)
        temp_history.append(T)
        T *= alpha

results = {
    "best_facilities": best,
    "best_cost": best_cost,
    "cost_history": cost_history,
    "temp_history": temp_history
}

return results

```

A.9 Funzione di costo k-center

```
def kcenter_cost(facilities_set, dist):
    n = len(dist)
    max_dist = 0.0

    for i in range(n):
        min_d = min(dist[i][j] for j in facilities_set)
        if min_d > max_dist:
            max_dist = min_d

    return max_dist
```

A.10 Visualizzazione della soluzione

```
def plot_solution(facilities_set, cities, dist, title="Facility Location"):
    n = len(cities)
    populations = [c[4] for c in cities]

    # assegna ogni citta' alla struttura piu' vicina
    assignment = []
    for i in range(n):
        nearest = min(facilities_set, key=lambda j: dist[i][j])
        assignment.append(nearest)

    # colori per i bacini di utenza
    facility_list = sorted(facilities_set)
    colors = plt.cm.tab20.colors

    fig, ax = plt.subplots(figsize=(10, 12))

    # disegna le citta' non-struttura
    for i in range(n):
        if i in facilities_set:
            continue
        color_idx = facility_list.index(assignment[i]) % len(colors)
        ax.plot(cities[i][3], cities[i][2], '.',
                color=colors[color_idx], markersize=4, alpha=0.6)

    # disegna le strutture
    for j in facilities_set:
        color_idx = facility_list.index(j) % len(colors)
        ax.plot(cities[j][3], cities[j][2], '*',
                color=colors[color_idx], markersize=15,
                markeredgewidth=0.5,
                label=cities[j][0])
```

```

ax.set_xlabel("longitudine")
ax.set_ylabel("latitudine")
ax.set_title(title)
ax.legend(loc='lower left', fontsize=6, ncol=2)
plt.tight_layout()
plt.show()

```

A.11 Curva del costo e della temperatura

```

def plot_sa_history(cost_history, temp_history):
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 6), sharex=True)

    ax1.plot(cost_history)
    ax1.set_ylabel("costo (km-persona)")
    ax1.set_title("Evoluzione del costo durante SA")

    ax2.semilogy(temp_history)
    ax2.set_xlabel("passo di raffreddamento")
    ax2.set_ylabel("temperatura T (scala log)")
    ax2.set_title("Cooling schedule")

    plt.tight_layout()
    plt.show()

```

A.12 Confronto tra cooling schedule

```

def compare_cooling_schedules(k, populations, dist,
                             alphas, n_runs=10):
    results = {}

    for alpha in alphas:
        costs = []
        for run in range(n_runs):
            res = simulated_annealing(
                k=k, populations=populations, dist=dist,
                T_start=1e7, T_end=1e2,
                alpha=alpha, steps_per_temp=50,
                seed=run
            )
            costs.append(res["best_cost"])

        results[alpha] = {
            "mean": statistics.mean(costs),
            "std": statistics.stdev(costs) if len(costs) > 1 else 0.0,
            "min": min(costs),

```

```

        "all_costs": costs
    }

    return results

def plot_cooling_comparison(results):
    labels = [str(a) for a in sorted(results.keys())]
    data = [results[a]["all_costs"] for a in sorted(results.keys())]

    plt.boxplot(data, labels=labels)
    plt.xlabel("alpha (cooling rate)")
    plt.ylabel("costo finale (km-persona)")
    plt.title("Confronto tra cooling schedule")
    plt.show()

```

A.13 Curva del costo ottimale in funzione di k

```

def optimal_cost_vs_k(k_values, populations, dist,
                      alpha=0.995, steps_per_temp=100):
    costs = []

    for k in k_values:
        res = simulated_annealing(
            k=k, populations=populations, dist=dist,
            T_start=1e7, T_end=1e2,
            alpha=alpha, steps_per_temp=steps_per_temp
        )
        costs.append(res["best_cost"])
        print(f"k={k}: costo = {res['best_cost']:.0f} km-persona")

    return costs

def plot_cost_vs_k(k_values, costs):
    marginal = [costs[i] - costs[i+1] for i in range(len(costs) - 1)]

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

    ax1.plot(k_values, costs, 'o-')
    ax1.set_xlabel("numero di strutture k")
    ax1.set_ylabel("costo ottimale (km-persona)")
    ax1.set_title("Costo ottimale vs k")

    ax2.bar(k_values[1:], marginal)
    ax2.set_xlabel("k")

```

```

ax2.set_ylabel("riduzione marginale del costo")
ax2.set_title("Rendimento marginale della k-esima struttura")

plt.tight_layout()
plt.show()

```

A.14 Esempio completo

```

if __name__ == "__main__":
    cities = CAPOLUOGHI
    n = len(cities)
    populations = [c[4] for c in cities]

    print(f"Calcolo matrice distanze ({n}x{n})...")
    dist = build_distance_matrix(cities)
    print("Fatto.")

    # run principale con k=10
    k = 10
    print(f"\nSimulated annealing con k={k}...")
    res = simulated_annealing(
        k=k, populations=populations, dist=dist,
        T_start=1e7, T_end=1e1,
        alpha=0.997, steps_per_temp=200,
        seed=42
    )

    print(f"Costo ottimale trovato: {res['best_cost']:.0f} km-persona")
    print("Strutture selezionate:")
    for j in sorted(res["best_facilities"]):
        print(f" {cities[j][0]} ({cities[j][1]})"
              f" - pop. {cities[j][4]:,}")

    plot_sa_history(res["cost_history"], res["temp_history"])
    plot_solution(res["best_facilities"], cities, dist,
                  title=f"Posizionamento ottimale di {k} ospedali")

    # curva del costo ottimale vs k
    k_values = [1, 2, 3, 5, 8, 10, 15, 20]
    costs = optimal_cost_vs_k(k_values, populations, dist)
    plot_cost_vs_k(k_values, costs)

```

A.15 Nota sulle unità di misura del costo

Il costo è espresso in **km-persona**: la distanza in km moltiplicata per il numero di abitanti. Ad esempio, se Milano (1.37 milioni di abitanti) è a 50 km dalla

struttura piu' vicina, il suo contributo al costo e' $1.37 \times 10^6 \times 50 = 6.85 \times 10^7$ km-persona.

Dividendo il costo totale per la popolazione totale si ottiene la **distanza media per abitante** in km, che e' la metrica piu' intuitiva per interpretare i risultati.

```
def average_distance_per_capita(cost, populations):  
    total_pop = sum(populations)  
    return cost / total_pop
```

A.16 Organizzazione consigliata del file

Per mantenere il codice leggibile, conviene organizzarlo in questo ordine:

1. import delle librerie;
2. dati: CAPOLUOGHI;
3. geografia:
 - haversine
 - build_distance_matrix
4. funzioni di costo:
 - kmedian_cost
 - kmedian_cost_delta
 - kcenter_cost
5. SA:
 - random_initial_solution
 - simulated_annealing
6. analisi:
 - compare_cooling_schedules
 - optimal_cost_vs_k
 - average_distance_per_capita
7. visualizzazione:
 - plot_solution
 - plot_sa_history
 - plot_cooling_comparison
 - plot_cost_vs_k
8. blocco finale con esempi.

A.17 Conclusione dell'appendice

La struttura proposta rende molto trasparente il funzionamento di SA: la funzione `simulated_annealing` implementa esattamente l'algoritmo descritto nella teoria, con il criterio di Metropolis visibile in tre righe di codice.

Il precalcolo della matrice delle distanze e' l'unica ottimizzazione non banale: senza di esso, ogni valutazione del costo richiederebbe di ricalcolare 108 distanze haversine, rendendo l'algoritmo troppo lento per l'analisi parametrica.

Il confronto diretto tra il costo della soluzione SA e quello della distribuzione reale degli ospedali — calcolato con la stessa funzione `kmedian_cost` — trasforma il laboratorio in un esercizio genuino di analisi di politica sanitaria con dati reali.